Homework 11

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from scipy.cluster.hierarchy import dendrogram, fcluster, linkage
from scipy.spatial.distance import pdist

pd.set_option("display.width", 120)
SEED = 432
rng = np.random.default_rng(SEED)
COLORS = np.array(["#1f77b4", "#d95f02"])
```

Question 1: Implementing K-means

K-means is a relatively simple algorithm that we can write ourselves. For Questions 1 and 2, do not use an existing function that performs K-means directly.

Generate 10 observations with two independent standard normal coordinates. Set a random seed and report it. Randomly assign the observations to $K = 2$ clusters, making sure neither cluster is empty. Let $C(i)$ denote the cluster assigned to observation x_i, and let m_k denote the mean of cluster k.

a. Perform one iteration: calculate the cluster means, then assign each observation to its closest mean using Euclidean distance. Report the means and updated assignments, and plot the observations colored by their updated assignments.

b. Write a function that repeats these two steps until the assignments stop changing or a specified maximum number of iterations is reached. Your function should work for a data matrix with any number of variables and a chosen number of clusters K. Start from a nonempty assignment to each cluster; if a cluster becomes empty during iteration, keep its previous mean. Recalculate the cluster means from the final assignments before returning them. Return the final assignments, cluster means, and within-cluster sum of squares,

$$\sum_{k=1}^{K} \sum_{i:\, C(i)=k} \|x_i - m_k\|^2.$$

c. Apply your function to the same 10 observations from two different random initial assignments. Report the final plots, cluster means, and within-cluster sums of squares. Do the two runs produce the same grouping and objective value? Explain your findings.

Setup. The random seed is 432, set once above and used for every random step below.

```
def random_assignment(n, K, rng):
    """Random labels 0..K-1 in which every cluster has at least one member."""
    order = rng.permutation(n)
    labels = rng.integers(K, size=n)
    labels[order[:K]] = np.arange(K)
    return labels


def cluster_means(X, labels, K, previous=None):
    """Mean of each cluster; an empty cluster keeps its previous mean."""
    means = np.full((K, X.shape[1]), np.nan) if previous is None else
previous.copy()
    for k in range(K):
        if np.any(labels == k):
            means[k] = X[labels == k].mean(axis=0)
    return means


def closest_mean(X, means):
    """Index of the closest mean (Euclidean distance) for each row of X."""
    squared = ((X[:, np.newaxis, :] - means[np.newaxis, :, :]) **
2).sum(axis=2)
    return squared.argmin(axis=1)


def plot_clusters(ax, X, labels, means, title):
    ax.scatter(X[:, 0], X[:, 1], c=COLORS[labels], s=55, edgecolor="white")
    ax.scatter(means[:, 0], means[:, 1], c=COLORS[: len(means)], marker="X",
s=190,
               edgecolor="black", label="cluster mean")
    for i, (x, y) in enumerate(X, start=1):
        ax.annotate(str(i), (x, y), xytext=(5, 4), textcoords="offset points",
fontsize=8)
    ax.set(xlabel="$x_1$", ylabel="$x_2$", title=title)
    ax.legend(loc="best", fontsize=8)


X = rng.standard_normal((10, 2))
initial = random_assignment(10, 2, rng)
pd.DataFrame(X, columns=["x1", "x2"], index=range(1,
11)).assign(initial=initial + 1).round(3)
```

	x1	x2	initial
1	0.708	−1.298	1
2	−1.352	−0.064	2
3	0.086	−0.583	2

	x1	x2	initial
4	−1.115	0.238	1
5	−0.486	1.806	1
6	−0.534	−1.988	1
7	2.127	−0.951	1
8	−0.812	0.358	2
9	0.278	0.778	1
10	0.725	−1.760	1

Part a.

```
means_one = cluster_means(X, initial, 2)
updated = closest_mean(X, means_one)

print(pd.DataFrame(means_one, columns=["x1", "x2"], index=["cluster 1",
"cluster 2"]).round(3))
print("Initial assignment:", (initial + 1).tolist())
print("Updated assignment:", (updated + 1).tolist())

fig, ax = plt.subplots(figsize=(5.5, 4.5))
plot_clusters(ax, X, updated, means_one, "One iteration")
plt.show()
```

```
              x1     x2
cluster 1  0.243 -0.453
cluster 2 -0.693 -0.096
Initial assignment: [1, 2, 2, 1, 1, 1, 1, 2, 1, 1]
Updated assignment: [1, 2, 1, 2, 2, 1, 1, 2, 1, 1]
```

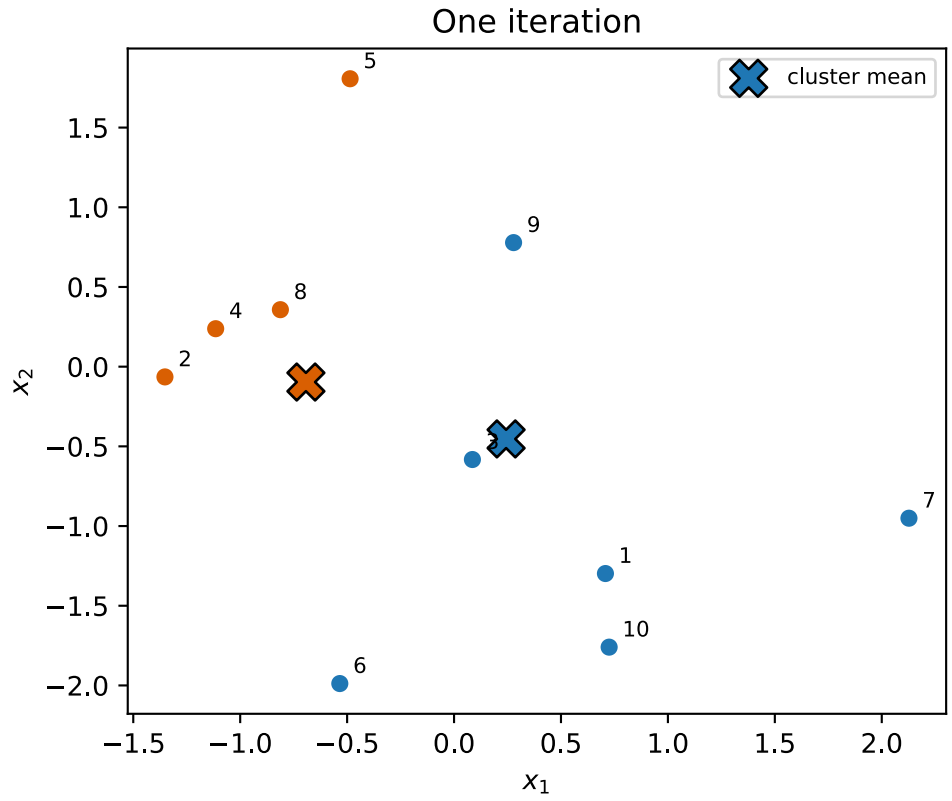


Figure 1: Observations after one K-means iteration, colored by updated assignment.

The table gives the means m_1, m_2 of the initial random clusters, which are the means used to make the updated assignment.

Part b.

```
def my_kmeans(X, K, initial, max_iter=100):
    """K-means from an initial assignment with labels 0..K-1."""
    X = np.asarray(X, dtype=float)
    labels = np.asarray(initial).copy()
    if labels.shape[0] != X.shape[0] or set(labels.tolist()) != set(range(K)):
        raise ValueError("initial must label every row and leave no cluster
empty")

    means = cluster_means(X, labels, K)
    converged = False
    iterations = 0
    while iterations < max_iter and not converged:
        iterations += 1
        means = cluster_means(X, labels, K, previous=means)
        new_labels = closest_mean(X, means)
        converged = np.array_equal(new_labels, labels)
```

```
        labels = new_labels

    means = cluster_means(X, labels, K, previous=means)
    wss = float(((X - means[labels]) ** 2).sum())
    return {"labels": labels, "means": means, "wss": wss,
            "iterations": iterations, "converged": converged}
```

The function works for any number of rows, variables, and clusters. It stops when an update changes no assignment or when `max_iter` is reached. An empty cluster keeps its previous mean, and it adds nothing to the within-cluster sum of squares.

Part c.

```
start_1 = random_assignment(10, 2, rng)
start_2 = random_assignment(10, 2, rng)
run_1 = my_kmeans(X, 2, start_1)
run_2 = my_kmeans(X, 2, start_2)

fig, axes = plt.subplots(1, 2, figsize=(10, 4.3), sharex=True, sharey=True)
for number, (ax, run) in enumerate(zip(axes, [run_1, run_2]), start=1):
    plot_clusters(ax, X, run["labels"], run["means"], f"Run {number}: WSS =
{run['wss']:.3f}")
plt.tight_layout()
plt.show()

for number, (start, run) in enumerate(zip([start_1, start_2], [run_1, run_2]),
start=1):
    print(f"Run {number}")
    print("  start: ", (start + 1).tolist())
    print("  final: ", (run["labels"] + 1).tolist())
    print("  means: ", run["means"].round(3).tolist())
    print(f"  WSS:    {run['wss']:.4f}   iterations: {run['iterations']}
converged: {run['converged']}")

same_grouping = (np.array_equal(run_1["labels"], run_2["labels"])
                 or np.array_equal(run_1["labels"], 1 - run_2["labels"]))
print("Same grouping, ignoring label names:", same_grouping)
```

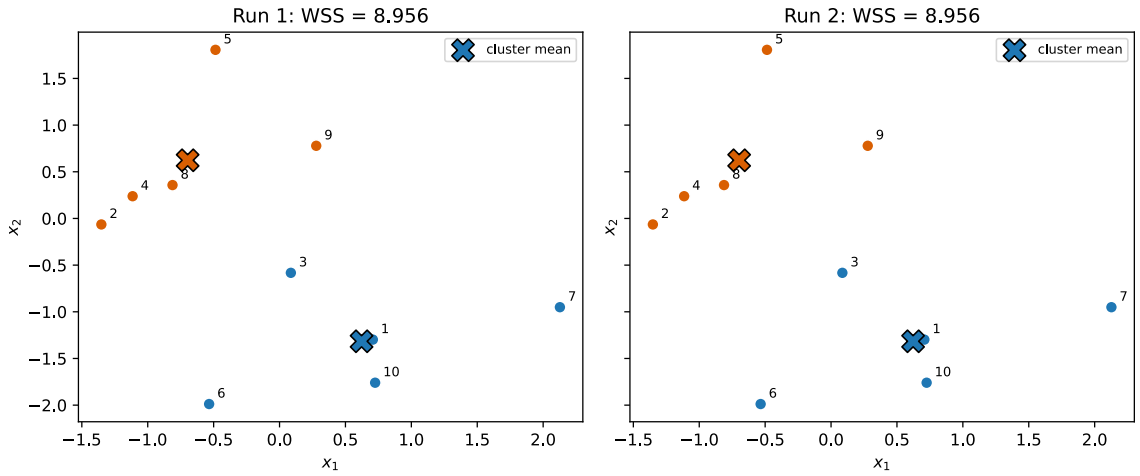


Figure 2: Final K-means clusters from two different random initial assignments.

```
Run 1
  start:  [1, 2, 1, 2, 1, 2, 1, 2, 2, 1]
  final:  [1, 2, 1, 2, 2, 1, 1, 2, 2, 1]
  means:  [[0.622, -1.316], [-0.697, 0.623]]
  WSS:    8.9557   iterations: 2   converged: True
Run 2
  start:  [2, 2, 1, 2, 1, 2, 1, 2, 1, 1]
  final:  [1, 2, 1, 2, 2, 1, 1, 2, 2, 1]
  means:  [[0.622, -1.316], [-0.697, 0.623]]
  WSS:    8.9557   iterations: 4   converged: True
Same grouping, ignoring label names: True
```

```python
# How common is each local solution across many random starts?
many = [my_kmeans(X, 2, random_assignment(10, 2, rng)) for _ in range(1000)]
outcomes = pd.DataFrame({
    "WSS": [round(run["wss"], 4) for run in many],
    "cluster without observation 1": [
        str((np.flatnonzero(run["labels"] != run["labels"][0]) + 1).tolist())
for run in many
    ],
})
outcomes.value_counts().sort_index().reset_index(name="starts out of 1000")
```

	WSS	cluster without observation 1	starts out of 1000
0	8.9557	[2, 4, 5, 8, 9]	947
1	17.3414	[5, 7, 9]	53

Yes. Both runs produced clusters $\{1, 3, 6, 7, 10\}$ and $\{2, 4, 5, 8, 9\}$, with means $(0.622, -1.316)$ and $(-0.697, 0.623)$ and within-cluster sum of squares 8.9557. They took different paths to get there: 2 update passes for Run 1 and 4 for Run 2, counting the last pass that changed nothing.

Agreement is not guaranteed. Each mean update and each reassignment can only lower the objective or leave it unchanged, so the algorithm stops at a local optimum that depends on the starting assignment. The check over 1000 more random starts found two stable solutions: 947 starts reached 8.9557, and 53 stopped at the worse value 17.3414, with $\{5, 7, 9\}$ split from the other seven observations. My two runs both reached the better solution, but in general K-means should be run from several starts, keeping the fit with the smallest objective.

Question 2: Image color compression

Pick your favorite color image and use your K-means function from Question 1 to cluster its pixels. For computational efficiency, resize the image to no more than 200×200 pixels and use its three RGB color channels.

a. Plot the original image and report its dimensions. Transform it into a data matrix with one observation per pixel and three variables: red, green, and blue. Report the dimensions of this matrix.

b. For $K = 2, 3, 5, 10$, apply your function to the pixel data. Report the cluster means and within-cluster sum of squares for each K. Reconstruct and plot each compressed image by replacing every pixel with its assigned cluster mean. Use the same RGB scale for all fits.

c. Which value of K would you choose for this image? Briefly justify your choice using the reconstructed images and the within-cluster sums of squares.

```
IMAGE_PATH = "data/portrait.png"  # change this one line to use a different
image

original = Image.open(IMAGE_PATH).convert("RGB")
small = original.copy()
small.thumbnail((200, 200), Image.Resampling.LANCZOS)
image = np.asarray(small, dtype=float) / 255  # RGB scale [0, 1] for every fit
height, width, channels = image.shape
pixels = image.reshape(-1, 3)

print("Original file size (height, width):", original.size[::-1])
print("Resized image array (height, width, channels):", image.shape)
print("Pixel data matrix (observations, variables):", pixels.shape)
print("Pixel values range from", pixels.min(), "to", pixels.max())
```

```
Original file size (height, width): (525, 500)
Resized image array (height, width, channels): (200, 190, 3)
Pixel data matrix (observations, variables): (38000, 3)
Pixel values range from 0.0 to 0.9333333333333333
```

Part a. The image is a portrait photo from https://stat.illinois.edu/directory/profile/rqzhu.

```
fig, ax = plt.subplots(figsize=(4.5, 4.9))
ax.imshow(image, vmin=0, vmax=1)
ax.set_title(f"Original ({height} x {width} pixels)")
ax.axis("off")
plt.show()
```

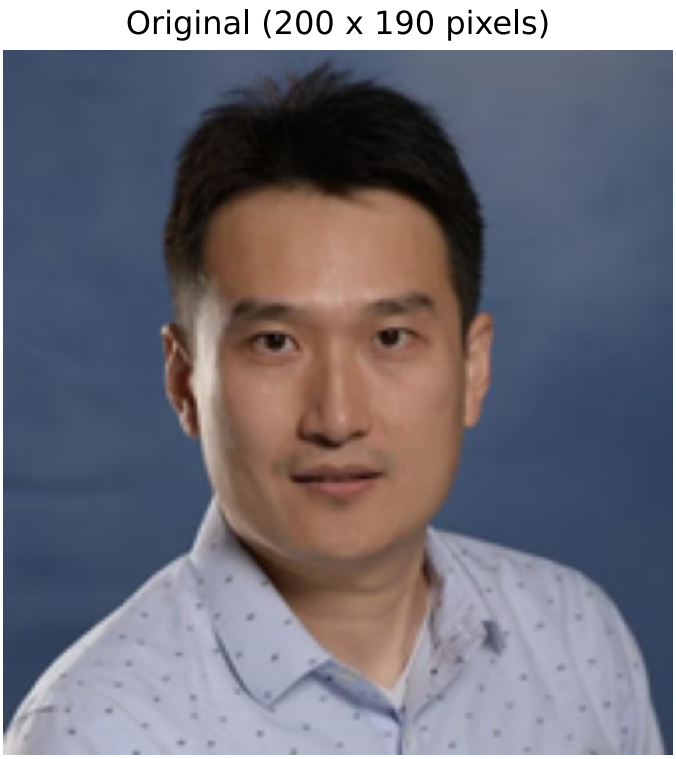


Figure 3: Original image after resizing.

Part b. Every fit uses the same pixel matrix with channels on the $[0, 1]$ scale. Random partitions can stop at different local optima, so for each K I ran my function from 5 random initial assignments and kept the fit with the smallest within-cluster sum of squares.

```
def best_of_starts(X, K, rng, starts=5, max_iter=1000):
    fits = [my_kmeans(X, K, random_assignment(len(X), K, rng), max_iter) for _
in range(starts)]
    return min(fits, key=lambda fit: fit["wss"]), all(fit["converged"] for fit
in fits)


Ks = [2, 3, 5, 10]
fits = {}
all_converged = {}
```

```
for K in Ks:
    fits[K], all_converged[K] = best_of_starts(pixels, K, rng)

total_ss = float(((pixels - pixels.mean(axis=0)) ** 2).sum())
summary = pd.DataFrame({
    "K": Ks,
    "WSS": [fits[K]["wss"] for K in Ks],
    "pct_of_total_SS_explained": [100 * (1 - fits[K]["wss"] / total_ss) for K
in Ks],
    "iterations_of_best_start": [fits[K]["iterations"] for K in Ks],
    "all_starts_converged": [all_converged[K] for K in Ks],
}).round(2)
print(f"Total sum of squares about the overall mean: {total_ss:.2f}")
summary
```

```
Total sum of squares about the overall mean: 3902.75
```

	K	WSS	pct_of_total_SS_explained	iterations_of_best_start	all_starts_converged
0	2	1461.38	62.55	10	True
1	3	883.06	77.37	11	True
2	5	287.51	92.63	18	True
3	10	103.84	97.34	28	True

```
for K in Ks:
    counts = np.bincount(fits[K]["labels"], minlength=K)
    table = pd.DataFrame(fits[K]["means"], columns=["red", "green", "blue"],
                         index=pd.RangeIndex(1, K + 1,
name="cluster")).round(3)
    table["pixels"] = counts
    print(f"K = {K}: WSS = {fits[K]['wss']:.2f}")
    print(table.to_string(), end="\n\n")
```

```
K = 2: WSS = 1461.38
           red  green   blue  pixels
cluster
1        0.660  0.603  0.617   12216
2        0.287  0.302  0.362   25784

K = 3: WSS = 883.06
           red  green   blue  pixels
cluster
1        0.115  0.088  0.074    3165
2        0.663  0.612  0.630   11584
```

```
3        0.318  0.335  0.402   23251

K = 5: WSS = 287.51
            red  green   blue  pixels
cluster
1        0.667  0.670  0.734    7628
2        0.110  0.085  0.072    3093
3        0.477  0.342  0.273    3520
4        0.272  0.327  0.428   18655
5        0.640  0.486  0.414    5104

K = 10: WSS = 103.84
            red  green   blue  pixels
cluster
1        0.614  0.449  0.366    3283
2        0.598  0.600  0.663    2331
3        0.700  0.534  0.455    1882
4        0.351  0.394  0.481    4574
5        0.083  0.067  0.056    2537
6        0.283  0.202  0.166    1036
7        0.263  0.322  0.427    9170
8        0.689  0.693  0.759    5485
9        0.221  0.280  0.385    4998
10       0.485  0.349  0.278    2704
```

```
fig, axes = plt.subplots(2, 3, figsize=(10, 7.2))
axes = axes.ravel()
axes[0].imshow(image, vmin=0, vmax=1)
axes[0].set_title("Original")
for ax, K in zip(axes[1:], Ks):
    compressed = fits[K]["means"][fits[K]["labels"]].reshape(height, width, 3)
    ax.imshow(compressed, vmin=0, vmax=1)
    ax.set_title(f"K = {K}, WSS = {fits[K]['wss']:.1f}")
axes[5].plot(Ks, [fits[K]["wss"] for K in Ks], marker="o", color="#1f77b4")
axes[5].set(xlabel="K", ylabel="Within-cluster SS", xticks=Ks, title="WSS by
K", ylim=(0, None))
for ax in axes[:5]:
    ax.axis("off")
plt.tight_layout()
plt.show()
```

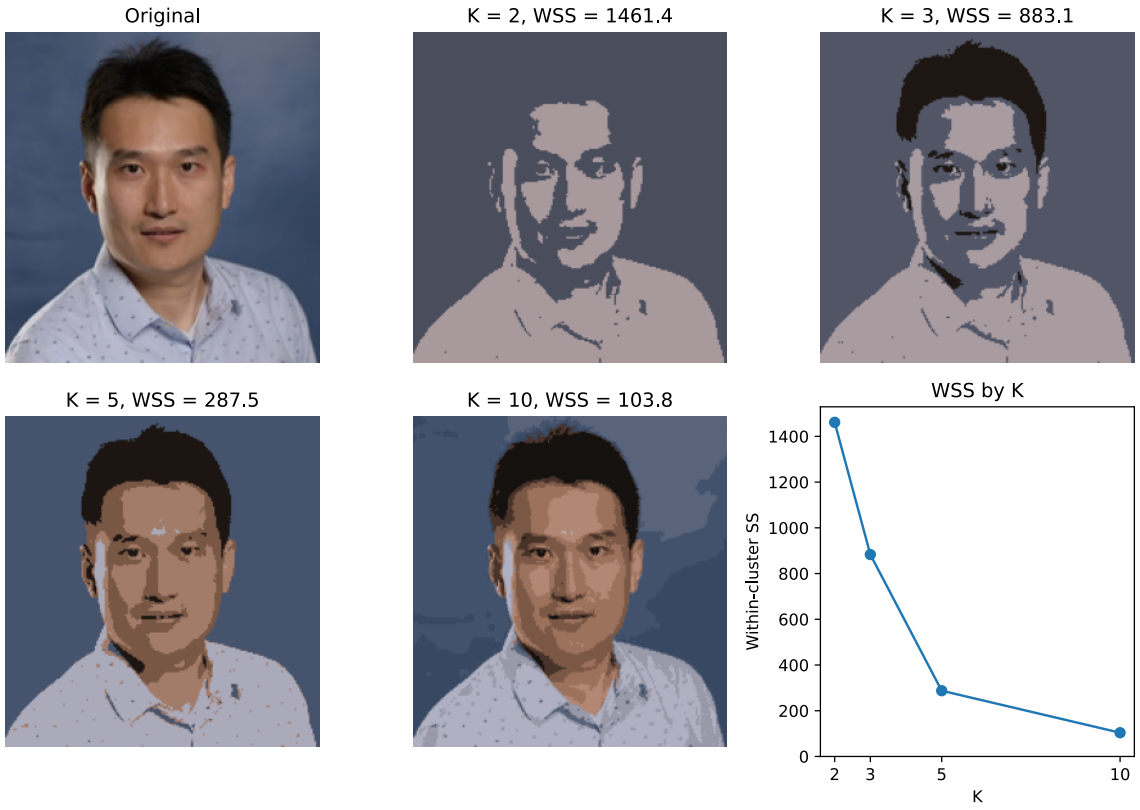


Figure 4: Original image and K-means reconstructions on a common RGB scale.

Part c. I would choose $K = 5$. The within-cluster sum of squares falls as K grows, so the useful question is where the improvement slows. Going from $K = 3$ to $K = 5$ reduces it from 883.06 to 287.51, about 298 per added cluster. Going from $K = 5$ to $K = 10$ reduces it from 287.51 to 103.84, only about 37 per added cluster, and $K = 5$ already explains 92.6% of the total sum of squares.

The images agree. $K = 2$ only separates light regions from dark regions, and $K = 3$ adds near-black hair, but neither has a realistic skin color. $K = 5$ gives the background, the hair, the shirt, and two skin tones, so the portrait looks natural and is easy to recognize. $K = 10$ does look closer to the original, with smoother shading on the face and visible variation in the background, so it would be the better choice if image quality mattered more than storing a small palette.

Question 3: Hierarchical clustering of cars

The file `data/mtcars.csv` contains 11 numerical variables for 32 cars, including fuel economy, horsepower, and weight. The `car` column gives the model names; use it to label observations, not as a clustering variable. You may use existing hierarchical clustering functions for this question.

a. Standardize each numerical variable to have mean 0 and standard deviation 1. Using Euclidean distance, perform hierarchical clustering with single, complete, and average linkage. Plot the three dendrograms with the car names and compare the results.

b. Choose one linkage method and a number of clusters that you consider reasonable. Report the cluster memberships and a table of cluster means on the original variable scales. Describe the main characteristics of the groups and explain your choice using the dendrogram and table. Briefly explain why standardizing the variables matters for this analysis.

```
cars = pd.read_csv("data/mtcars.csv").set_index("car")
scaled = (cars - cars.mean()) / cars.std(ddof=1)
distances = pdist(scaled.to_numpy(), metric="euclidean")
methods = ["single", "complete", "average"]
trees = {method: linkage(distances, method=method) for method in methods}
print("Data:", cars.shape, "  scaled means near 0:",
bool(np.allclose(scaled.mean(), 0)),
      "  scaled SDs equal 1:", bool(np.allclose(scaled.std(ddof=1), 1)))
```

```
Data: (32, 11)   scaled means near 0: True   scaled SDs equal 1: True
```

Part a.

```
fig, axes = plt.subplots(1, 3, figsize=(10.5, 9))
for ax, method in zip(axes, methods):
    dendrogram(trees[method], labels=cars.index.tolist(), orientation="left",
ax=ax,
               color_threshold=0, above_threshold_color="#333333",
leaf_font_size=8)
    ax.set(title=f"{method.capitalize()} linkage", xlabel="Merge height")
plt.tight_layout()
plt.show()
```

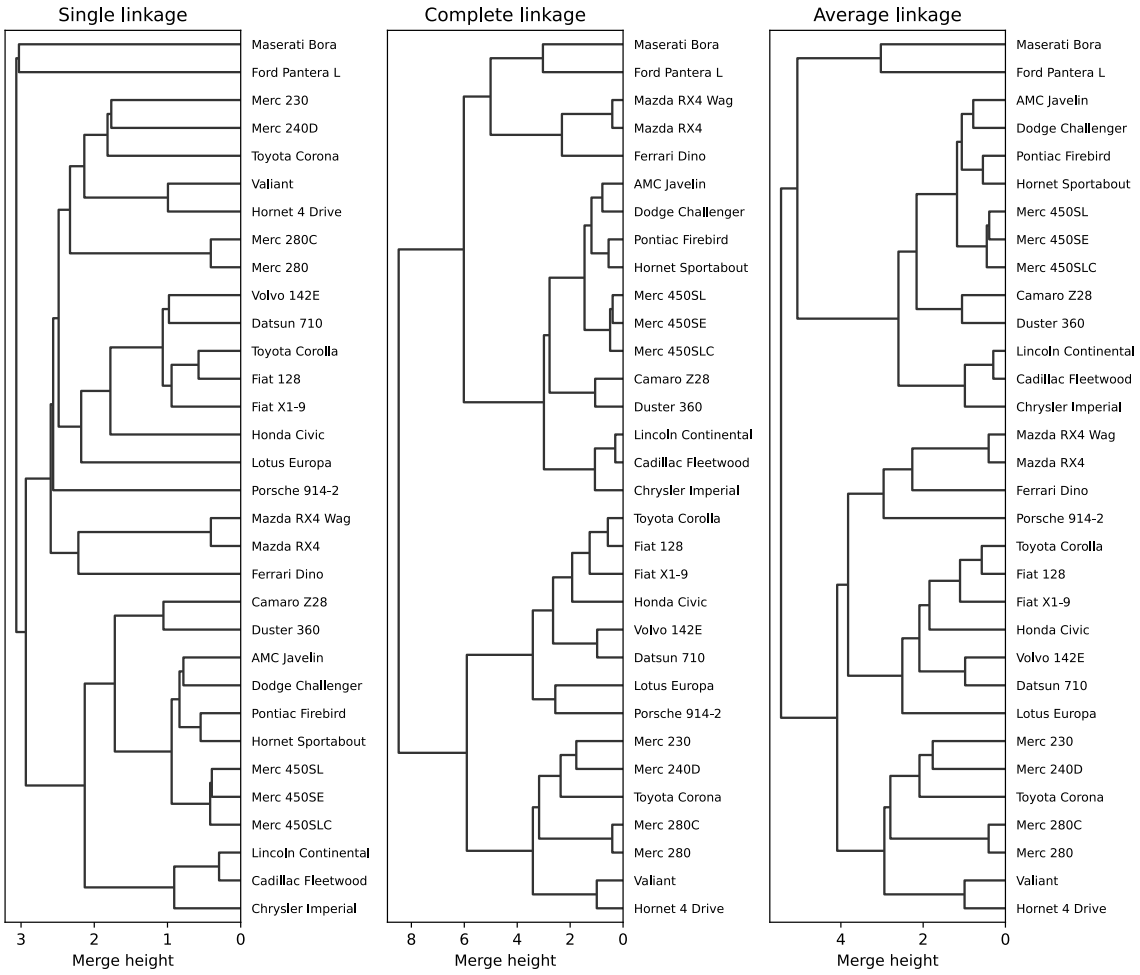


Figure 5: Dendrograms of the standardized car data under three linkage methods.

```
pd.DataFrame({
    method: {
        "largest cluster when cut to 2":
int(np.bincount(fcluster(trees[method], 2, "maxclust")).max()),
        "largest cluster when cut to 3":
int(np.bincount(fcluster(trees[method], 3, "maxclust")).max()),
        "largest cluster when cut to 4":
int(np.bincount(fcluster(trees[method], 4, "maxclust")).max()),
        "final merge height": round(float(trees[method][-1, 2]), 2),
    }
    for method in methods
})
```

	single	complete	average
largest cluster when cut to 2	30.00	17.00	18.00

	single	complete	average
largest cluster when cut to 3	30.00	15.00	18.00
largest cluster when cut to 4	18.00	12.00	12.00
final merge height	3.06	8.48	5.45

The three linkages agree on the closest cars: the Merc 450 models, Lincoln Continental with Cadillac Fleetwood, Merc 280 with 280C, and Mazda RX4 with RX4 Wag join early in every tree. They differ in the large-scale structure.

- **Single linkage** shows chaining. Merge heights are compressed between about 2.5 and 3.06 near the top, and Ford Pantera L and Maserati Bora join last, so cutting into 2 or 3 clusters leaves a 30-car cluster.
- **Complete linkage** produces the most compact and balanced groups, with 15 and 17 cars at two clusters and a clear final merge at 8.48.
- **Average linkage** falls in between. It finds broad groups, but Ford Pantera L and Maserati Bora stay apart until a height of 5.06, so three clusters have sizes 18, 12, and 2.

Part b.

```
NUMBER_OF_CLUSTERS = 3
groups = pd.Series(fcluster(trees["complete"], NUMBER_OF_CLUSTERS,
"maxclust"),
                   index=cars.index, name="cluster")
print("Merge heights of the last five complete-linkage joins:",
      trees["complete"][-5:, 2].round(2).tolist())
for number, members in groups.groupby(groups):
    print(f"\nCluster {number} ({len(members)} cars): " + ",
".join(members.index))
```

```
Merge heights of the last five complete-linkage joins: [3.41, 5.01, 5.9, 6.02,
8.48]

Cluster 1 (15 cars): Datsun 710, Hornet 4 Drive, Valiant, Merc 240D, Merc 230,
Merc 280, Merc 280C, Fiat 128, Honda Civic, Toyota Corolla, Toyota Corona,
Fiat X1-9, Porsche 914-2, Lotus Europa, Volvo 142E

Cluster 2 (12 cars): Hornet Sportabout, Duster 360, Merc 450SE, Merc 450SL,
Merc 450SLC, Cadillac Fleetwood, Lincoln Continental, Chrysler Imperial, Dodge
Challenger, AMC Javelin, Camaro Z28, Pontiac Firebird

Cluster 3 (5 cars): Mazda RX4, Mazda RX4 Wag, Ford Pantera L, Ferrari Dino,
Maserati Bora
```

```
cars.groupby(groups).mean().round(2).assign(n=groups.value_counts().sort_index())
```

cluster	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb	n
1	24.65	4.53	131.65	91.33	3.90	2.58	19.16	0.93	0.53	3.93	1.80	15
2	15.05	8.00	357.62	194.17	3.12	4.10	17.14	0.00	0.00	3.00	3.08	12
3	18.50	6.80	223.40	198.80	3.84	3.00	15.62	0.00	1.00	4.60	5.20	5

I chose complete linkage with 3 clusters, because it gives compact groups of useful size and a clear interpretation.

- **Cluster 1, economy cars (15 cars):** the highest fuel economy (24.65 mpg), the smallest engines (132 cubic inches, 4.5 cylinders on average), low power (91 hp), and low weight (2.58 thousand lb). Nearly all have straight engines (`vs` 0.93).
- **Cluster 2, large V8 cars (12 cars):** the lowest fuel economy (15.05 mpg), 8 cylinders in every car, the largest engines (358 cubic inches), high power (194 hp), and the highest weight (4.10 thousand lb). All are automatic with 3 gears.
- **Cluster 3, sporty manual cars (5 cars):** power like Cluster 2 (199 hp) at much lower weight (3.00 thousand lb). They have the quickest quarter-mile times (15.62 s), and all are manual, averaging 4.6 gears and 5.2 carburetors.

The dendrogram supports this choice only in part. The widest gap between merge heights is from 6.02 to 8.48, which favors 2 clusters. I used 3 because the 6.02 merge joins two groups that the table shows are very different: heavy automatic V8 cars and lighter manual sports cars. The cut itself is delicate, though, since 3 clusters exist only for heights between 5.90 and 6.02. The next split, at 5.90, divides Cluster 1 into a group of 7 and a group of 8, so a 4-cluster solution would be nearly as well supported by the heights.

Standardizing matters because Euclidean distance adds squared differences, so variables with large numerical ranges would otherwise dominate. On the original scales, `disp` and `hp` together account for 99.8% of the total variance computed below. The clustering would then mostly reflect those two variables and their units. Standardizing puts every variable on a comparable scale, so differences in weight, gears, transmission, and the other variables can also affect the groups.

```
variances = cars.var(ddof=1)
(100 * variances /
variances.sum()).round(2).sort_values(ascending=False).rename("% of total
variance").to_frame()
```

	% of total variance
disp	76.39
hp	23.38
mpg	0.18

	% of total variance
cyl	0.02
qsec	0.02
carb	0.01
drat	0.00
wt	0.00
vs	0.00
am	0.00
gear	0.00